

Piastrelle colorate

Barchin e Charos stanno giocando a piastrellare una griglia di $2N \times 2M$ celle. Le righe sono numerate da 0 a $2N - 1$ dall'alto verso il basso, e le colonne sono numerate da 0 a $2M - 1$ da sinistra a destra. Per $0 \leq i < 2N$ e $0 \leq j < 2M$, indichiamo con (i, j) la cella nella riga i e nella colonna j .

Barchin dà a Charos $N \cdot M$ **piastrelle**, una alla volta. Ogni piastrella è un quadrato di 2×2 **sezioni**, ciascuna grande come una cella. Barchin colora ogni sezione di nero o bianco, garantendo che **almeno una sezione sia bianca**.

Charos deve posizionare ogni piastrella sulla griglia **subito dopo averla ricevuta**, senza sapere quali piastrelle riceverà in seguito. Le piastrelle non possono essere ruotate, devono essere piazzate interamente dentro la griglia, e non possono sovrapporsi. Inoltre, la sezione in alto a sinistra di ogni piastrella deve coprire esattamente una cella le cui coordinate siano entrambe pari.

Barchin vince la partita se, dopo che una piastrella è stata posizionata, esiste un quadrato di 2×2 celle coperte da quattro sezioni nere. Formalmente, se le celle (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ sono tutte coperte da sezioni nere per qualche $0 \leq a < 2N - 1$ e $0 \leq b < 2M - 1$, allora Barchin vince. Le quattro sezioni nere possono appartenere a piastrelle diverse, e gli indici a e b non devono necessariamente essere pari.

Charos vince se piazza tutte le $N \cdot M$ piastrelle, coprendo completamente la griglia, senza che Barchin vinca mai. Implementa una strategia che permetta a Charos di vincere la partita!

Si può dimostrare che, con le assunzioni sopra, Charos ha sempre una strategia per posizionare le piastrelle garantendosi la vittoria, indipendentemente da quello che riceverà in seguito.

Implementazione

Devi implementare due funzioni:

```
void init(int N, int M)
```

- N : metà del numero di righe della griglia.
- M : metà del numero di colonne della griglia.
- La funzione viene chiamata esattamente una volta per caso di test, all'inizio dell'esecuzione del programma.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : i colori delle sezioni in alto a sinistra, in alto a destra, in basso a sinistra e in basso a destra della piastrella corrente, come mostrato nella figura sottostante. Ogni valore è 0 (bianco) o 1 (nero).
- La funzione viene chiamata esattamente $N \cdot M$ volte per caso di test, dopo `init`.



La funzione deve restituire una coppia (i, j) , dove i è la riga e j è la colonna della cella in cui deve essere posizionata la sezione in alto a sinistra di questa piastrella. Sia i che j devono essere pari e la regione 2×2 coperta dalla piastrella non deve essere già coperta da un'altra piastrella.

Se `receive_block` restituisce una coppia che non soddisfa questi requisiti, o se dopo aver posizionato la piastrella un quadrato di 2×2 celle viene completamente coperto da sezioni nere, il grader terminerà il programma restituendo `Output isn't correct` per il caso di test.

Il comportamento del grader **non è adattivo**: la sequenza di piastrelle che Barchin passa a Charos è fissata prima che venga chiamata `init`.

Assunzioni

Per ogni piastrella, sia S il numero delle sue sezioni nere, ovvero, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$.
- $0 \leq S \leq 3$ per ogni piastrella.

Subtask

Subtask	Punteggio	Assunzioni aggiuntive
1	6	$S = 1$ per ogni piastrella e $N = 2$.
2	16	$S = 3$ per ogni piastrella, N è pari e uguale a M , e ognuna delle quattro colorazioni con $S = 3$ compare esattamente $N^2/4$ volte.
3	10	$S = 1$ per ogni piastrella.
4	29	$S \leq 2$ per ogni piastrella.
5	39	Nessuna assunzione aggiuntiva.

Esempio

Siano $N = 1$ e $M = 2$, per cui la griglia ha 2 righe e 4 colonne. Il grader chiama prima:

```
init(1, 2)
```

Inizialmente, tutte le celle sono vuote:

	0	1	2	3
0				
1				

Dobbiamo piazzare $N \cdot M = 2$ piastrelle. Supponiamo che Barchin dia una piastrella con tre sezioni nere e una bianca nell'angolo in alto a destra. Il grader chiama:

```
receive_block(1, 0, 1, 1)
```

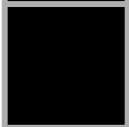
Charos posiziona questa piastrella sul lato sinistro della griglia restituendo $(0,0)$. La griglia ora si presenta così:

	0	1	2	3
0				
1				

Barchin aggiunge quindi una piastrella con tre sezioni nere e una bianca nell'angolo in alto a sinistra:

```
receive_block(0, 1, 1, 1)
```

L'unica cella valida come angolo in alto a sinistra di una piastrella 2×2 è $(0, 2)$, quindi Charos restituisce $(0, 2)$. La griglia rimane quindi:

	0	1	2	3
0				
1				

Nessun quadrato 2×2 è completamente coperto da sezioni nere, quindi Charos ha posizionato con successo tutte le piastrelle senza che Barchin vincesses mai, e vince la partita.

Grader di esempio

Formato di input:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato di output:

```
R[0] C[0]
R[1] C[1]
...
R[NM-1] C[NM-1]
```

Dove $R[k]$ e $C[k]$ sono gli interi restituiti dalla k -esima chiamata a `receive_block`.